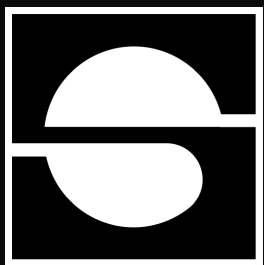




Security Assessment Report



Saturn Dollar M0 Extensions

March 2026

Prepared for Saturn

Table of Contents

Project Summary.....	3
Project Scope.....	3
Project Overview.....	3
Protocol Overview.....	3
Assessment Methodology.....	5
Threat and Security Overview.....	6
Findings Summary.....	7
Severity Matrix.....	7
Detailed Findings.....	8
High Severity Issues.....	10
H-01: Supply cap check uses mismatched decimals (can incorrectly block wraps or make cap meaningless).....	10
H-02: MIN_WITHDRAWAL threshold is incompatible with USDat decimals, permanently blocking all redemptions.....	12
Medium Severity Issues.....	14
M-01: Supply Cap Can Be Bypassed Via claimYield() Since Yield Minting Ignores Supply-cap Checks.....	14
Low Severity Issues.....	15
L-01: Whitelist checks are bypassed for JMI asset wraps.....	15
L-02: WithdrawalQueueERC721.addRequest uses _mint instead of _safeMint.....	16
Informational Severity Issues.....	18
I-01: Compliance role cannot transfer its own NFTs.....	18
I-02: Hardcoded price decimals in validation functions.....	19
I-03: Paused USDat Could Lead To Unbacked `USDat` in the Staking Contract When `STRC` Is Purchased.....	20
I-04: Redundant zero-address validation in USDat.initialize.....	21
Disclaimer.....	23
About Certora.....	23

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Saturn Dollar MO Extensions	https://github.com/saturn-organization/saturn-yield-dollar	c5b19e5 (audit) 9c98678 (fix)	Solidity
	https://github.com/saturn-organization/saturn-dollar	e8d167c (audit) 11582ff (fix)	

Project Overview

This document describes the security review of **Saturn Dollar MO Extensions**. The work was undertaken from **February 23rd, 2026** to **February 27th, 2026**.

The following contract list is included in our scope:

```
saturn-organization/saturn-yield-dollar/src/*  
saturn-organization/saturn-dollar/src/*
```

The team performed a manual audit of all the files in scope. During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

Saturn is a DeFi protocol that issues a USD-denominated stablecoin (USDat) and a yield-bearing vault token (sUSDat). USDat is built on MO's M token infrastructure as a JMIExtension, minted 1:1 against stable assets by KYC-verified users.

Users can permissionlessly deposit USDat into the sUSDat vault (an ERC4626-style contract) to gain synthetic exposure to STRC MicroStrategy's NASDAQ-listed preferred equity, which trades at approximately \$100 and pays ~11% monthly cash dividends. STRC is tracked entirely on-chain via a virtual balance (`strcBalance`); no ERC20 token exists. A trusted entity buys and sells real



STRC off-chain through a brokerage account and reconciles the on-chain state via `convertFromUsdat` and `convertFromStrc`, which are subject to oracle-validated tolerance checks.

STRC dividends are distributed to sUSDat holders through `transferInRewards` with linear vesting over a configurable period (default 30 days). The vault's `totalAssets()` is denominated in USD and reflects on-chain USDat, STRC balance corresponding to the STRC purchased by the offchain entity and the USD value of vested STRC priced via a Chainlink-compatible oracle.

Withdrawals from sUSDat are asynchronous and handled through a dedicated `WithdrawalQueue` implemented as an ERC721 contract. Users submit redemption requests that escrow shares into the queue and specify a minimum USDat to receive. The processor locks, processes, and fulfills requests by selling STRC off-chain and providing USDat. Processed requests can then be claimed by users. Requests cannot be cancelled, and compliance can seize blacklisted positions.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

The primary security invariant is that `totalAssets() = usdatBalance + vestedStrcValue` must accurately reflect the protocol's backing at all times, and that shares burned during withdrawal processing correspond to value actually realized off-chain and reflected on-chain. Because settlement is delayed and execution occurs off-chain, correctness relies on enforcing these invariants through on-chain validation rather than atomic execution.

The most significant trust boundary is the `PROCESSOR_ROLE`, which supplies critical settlement parameters and controls conversion timing. While the protocol includes oracle checks, tolerance bounds (default 20%), vesting constraints, and validation logic to reduce reliance on processor honesty, the processor remains capable of influencing economic outcomes. Additional trusted roles include the Admin (pause, upgrades, parameters) and Compliance (blacklist, freeze, seize).

The audit considered attack vectors including:

- Decimal mismatches between `USDat` (6 decimals), vault shares (18 decimals), and oracle prices (8 decimals)
- Oracle failure scenarios, staleness during NASDAQ weekend closures, price bound breaches
- Processor under-settlement while satisfying tolerance checks
- ERC4626 inflation and donation attacks (mitigated by `_decimalsOffset = 6`)
- Withdrawal queue state machine edge cases, stuck requests, NFT transfer to blacklisted addresses
- Compliance bypass via blacklisted spender allowances

All core contracts were manually reviewed, including `USDat`, `StakedUSDat`, `WithdrawalQueueERC721`, and `StrcPriceOracle`. Invariants around share supply, NAV computation, and withdrawal settlement were analyzed across contract boundaries rather than in isolation.

The repository does not include test coverage for `StakedUSDat` or the associated withdrawal and processing modules. We recommend adding comprehensive tests covering all important scenarios to improve confidence in the protocol's correctness.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	–	–	–
High	2	2	2
Medium	1	1	1
Low	2	2	2
Informational	4	4	3
Total	10	10	9

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
Low	Low	Low	Low	Medium
	Low	Medium	High	
Likelihood				

Detailed Findings

ID	Title	Severity	Status
H-01	Supply cap check uses mismatched decimals (can incorrectly block wraps or make cap meaningless)	High	Fixed
H-02	MIN_WITHDRAWAL threshold is incompatible with USDat decimals, permanently blocking all redemptions	High	Fixed
M-01	Supply Cap Can Be Bypassed Via `claimYield()` Since Yield Minting Ignores Supply-cap Checks)	Medium	Fixed
L-01	Whitelist checks are bypassed for JMI asset wraps	Low	Fixed
L-02	WithdrawalQueueERC721.addRequest uses `_mint` instead of `_safeMint`	Low	Fixed
I-01	Compliance role cannot transfer its own NFTs	Informational	Fixed
I-02	Hardcoded price decimals in validation functions	Informational	Fixed
I-03	Paused USDat Could Lead To Unbacked `USDat` in the Staking Contract When `STRC` Is Purchased	Informational	Acknowledged

[I-Q4](#)

Redundant zero-address validation in
USDat.initialize

Informational

Fixed

High Severity Issues

H-01: Supply cap check uses mismatched decimals (can incorrectly block wraps or make cap meaningless)

Severity: High	Impact: Medium	Likelihood: High
Files: saturn-dollar/USDat	Status: Fixed	

Description:

USDat enforces the supply cap in `_revertIfSupplyCapExceeded(amount)` by checking:

```
function _revertIfSupplyCapExceeded(uint256 amount) internal view {
    SupplyStorage storage $ = _getSupplyStorage();
    if ($.isEnabled && totalSupply() + amount > $.cap) {
        revert SupplyCapExceeded(totalSupply(), amount, $.cap);
    }
}
```

`totalSupply()` is denominated in USDat (extension) decimals which are 6 and amount is passed into `_beforeWrap(account, recipient, amount)` by MO, and for JMI wraps this amount is the input asset amount, denominated in the asset's own decimals which can be 18 decimals for assets like DAI.

So the cap check is comparing values in different units. It treats the input amount as if it were already USDat units.

Due to this enabling the supply cap can incorrectly block wraps (e.g., wrapping 1 DAI reverts) because the cap is checked in 6-decimal USDat units while the wrapped asset amount is 18-decimals.

Recommendations:

Ensure the cap and asset amount is represented in the same decimal units.

Customer Response:



Fixed in [4e750b3](#).

Fix Review:

Confirmed.

H-02: MIN_WITHDRAWAL threshold is incompatible with USDat decimals, permanently blocking all redemptions

Severity: High	Impact: High	Likelihood: High
Files: saturn-yield-dollar/src/Stake dUSDat.sol	Status: Fixed	

Description:

`StakedUSDat` defines a minimum withdrawal constant of `10e18`:

```
/// @notice Minimum withdrawal amount (10 USDat)
uint256 public constant MIN_WITHDRAWAL = 10e18;
```

Users withdraw from `StakedUSDat` by calling `requestRedeem(shares)`, which computes the corresponding asset amount via `previewRedeem(shares)` and passes it to `_processWithdrawal`, which function enforces a minimum:

```
function _processWithdrawal(...) internal ... returns (uint256 requestId) {
    ...
    require(assets >= MIN_WITHDRAWAL, WithdrawalTooSmall());
    ...
}
```

However, `previewRedeem` uses the ERC4626 `_convertToAssets` formula, which returns values denominated in the underlying asset's decimals. USDat uses 6 decimals, so `previewRedeem` always returns a 6-decimal value. For a deposit of 1,000,000 USDat (`1_000_000e6`), redeeming all resulting shares returns `assets ≈ 1e12` — six orders of magnitude below `MIN_WITHDRAWAL = 10e18`.

Note that `withdraw()` and `redeem()` are overridden to unconditionally revert, making `requestRedeem()` the sole withdrawal mechanism. With this path blocked, all deposited USDat is permanently locked in the vault.

All user funds deposited into `StakedUSDat` are irretrievable. Any call to `requestRedeem` will revert regardless of the number of shares being redeemed. Since this is the only withdrawal path available to users, the vault becomes a one-way deposit with no exit.

Recommendations:

Change `MIN_WITHDRAWAL` to a 6-decimal value consistent with USDat's precision:

```
- uint256 public constant MIN_WITHDRAWAL = 10e18;  
  
+ uint256 public constant MIN_WITHDRAWAL = 10e6;
```

Customer Response:

Fixed in [cbd0450](#).

Fix Review:

Confirmed.

Medium Severity Issues

M-01: Supply Cap Can Be Bypassed Via `claimYield()` Since Yield Minting Ignores Supply-cap Checks

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: saturn-dollar/USDat.sol	Status: Fixed	

Description:

The cap enforcement is applied in the wrapping path (`_beforeWrap()` → `_revertIfSupplyCapExceeded()`). However, in the M0 MYieldToOne design, `claimYield()` mints USDat directly to `yieldRecipient` using `_mint()` and does not go through `_beforeWrap()`.

So even when the supply cap is enabled, calling `claimYield()` can increase total USDat supply beyond the configured cap. This can lead to `totalSupply()` being greater than the supply cap, once `totalSupply()` exceeds the cap, all future wraps will revert.

Recommendations:

Enforce the supply cap on yield minting too (e.g., add a cap check in the `claimYield()` path via an override like `_beforeClaimYield()`)

Customer Response:

Fixed in [4e750b3](#).

Fix Review:

Confirmed.

Low Severity Issues

L-01: Whitelist checks are bypassed for JMI asset wraps

Severity: Low	Impact: Medium	Likelihood: Medium
Files: saturn-dollar/USDat	Status: Fixed	

Description:

USDat only overrides the 3-parameter `_beforeWrap(address account, address recipient, uint256 amount)` hook, but JMI asset wraps use JMIExtension implementation's `_beforeWrap(address asset, address account, address recipient, uint256 amount)`. During `wrap(asset, recipient, amount)`, the 4-parameter JMI hook is executed and its `super._beforeWrap(account, recipient, amount)` resolves to `MYieldToOne`, not USDat's 3-parameter override. As a result, USDat's whitelist checks are not enforced for external-asset wraps via the SwapFacility.

Recommendations:

Also override the 4-parameter JMI hook in `USDat` and enforce whitelist checks there.

Customer Response:

Fixed in [984f226](#).

Fix Review:

Confirmed.

L-02: WithdrawalQueueERC721.addRequest uses `_mint` instead of `_safeMint`

Severity: Low	Impact: Low	Likelihood: Low
Files: saturn-yield-dollar/src/WithdrawalQueueERC721.sol	Status: Fixed	

Description:

In `WithdrawalQueueERC721.addRequest`, the withdrawal request NFT is minted using `_mint`:

```
function addRequest(address user, uint256 shares, uint256 minUsdatReceived)
    external
    nonReentrant
    whenNotPaused
    onlyRole(STAKED_USDAT_ROLE)
    returns (uint256 tokenId)
{
    ...
    _mint(user, tokenId);

    emit WithdrawalRequested(tokenId, user, shares, block.timestamp);
}
```

Unlike `_safeMint`, `_mint` does not call `onERC721Received` on the recipient. If a `user` is a contract that does not implement the `IERC721Receiver` interface, the NFT will be minted successfully, but the contract will have no way to interact with it. Since the withdrawal NFT represents escrowed sUSDat shares and is required to claim the underlying USDat, this would result in the withdrawal being permanently stuck.

The risk is limited because `requestRedeem` in `StakedUSDat` passes `msg.sender` as the user, and a contract initiating the redemption is likely aware of ERC721 interactions. However, the ERC721 standard recommends `_safeMint` to protect against accidental token loss in contracts that lack ERC721 support.

Recommendations:

Replace `_mint` with `_safeMint` to ensure the recipient can handle ERC721 tokens.

Customer Response:

Fixed in [ceed44e](#).

Fix Review:

Confirmed.

Informational Severity Issues

I-01: Compliance role cannot transfer its own NFTs

Files:

saturn-yield-dollar/src/WithdrawalQueueERC721.sol

Description:

The `_update` override in `WithdrawalQueueERC721` assumes any transfer by a `COMPLIANCE_ROLE` holder is a seizure, enforcing `_requireBlacklisted(from)`. This prevents compliance officers from transferring their own legitimately owned NFTs, since they are not blacklisted.

Recommendations:

Only enforce the blacklist seizure check when `from != msg.sender`.

Customer Response:

Fixed in [d451cd3](#).

Fix Review:

Confirmed.

I-02: Hardcoded price decimals in validation functions

Files:

saturn-yield-dollar/src/WithdrawalQueueERC721.sol
saturn-yield-dollar/src/StakedUSDat.sol

Description:

`_validateConversion` (`StakedUSDat`) and `_validateTotals` (`WithdrawalQueueERC721`) hardcode price precision to `1e8`, while `_strcTotalAssets` dynamically fetches `priceDecimals` from the oracle. If `updateOracle()` switches to a feed with different precision, `totalAssets` calculation adapts, but the validation logic silently breaks.`

Recommendations:

Replace hardcoded `1e8` with `10 ** priceDecimals` fetched from the oracle.

Customer Response:

Fixed in [3a6398b](#).

Fix Review:

Confirmed.

I-03: Paused USDat Could Lead To Unbacked `USDat` in the Staking Contract When `STRC` Is Purchased

Files:

saturn-yield-dollar/StakedUSDat

Description:

In a scenario where the offchain entity purchases **STRC** from the market the **PROCESSOR_ROLE** is supposed to call the **convertFromUsdat()** to account for the **STRC** purchased , but if the **UDSat** contract is paused i.e. the JMI extension token contract is paused then **USDat** transfers would also be paused i.e. would revert , this could lead to a situation where the **STRC** is purchased by the entity from the market but **USDat** supply can not be decreased synchronously from the Staking contract (since the call to **convertFromUsdat()** would revert) which would lead to unbacked **USDat** in the staking contract corresponding to the **STRC** purchased from the market till the **USDat** contract is resumed back again.

Recommendations:

Consider acknowledging the risk and make sure to not purchase **STRC** when **USDat** transfers are paused.

Customer Response:

Acknowledged.

I-04: Redundant zero-address validation in USDat.initialize

Files:

saturn-dollar/src/USDat.sol

Description:

`USDat.initialize()` performs an upfront zero-address check on all four parameters:

```
function initialize(address yieldRecipient, address admin, address
compliance, address processor)
    public
    initializer
{
    if (yieldRecipient == address(0) || admin == address(0) || compliance ==
address(0) || processor == address(0))
    {
        revert ZeroAddress();
    }
    ...
}
```

This check is redundant, as all four addresses are individually validated downstream during the initialization chain:

- `yieldRecipient` — validated in `MYieldToOne._setYieldRecipient`
- `admin` — validated in `MYieldToOne.__MYieldToOne_init`
- `compliance` — validated as `freezeManager` in `Freezable` and as `pauser` in `Pausable`
- `processor` — validated as `assetCapManager` in `JMIExtension` and as `yieldRecipientManager` in `MYieldToOne`

The check increases deployment gas cost without adding any safety benefit.

Recommendations:

Remove the redundant zero-address check from `USDat.initialize()` and rely on the existing downstream validations.

Customer Response:



Fixed in [18543f7](#).

Fix Review:

Confirmed.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.